



华南理工大学
South China University of Technology

研究生课程论文

移动软件测试工具体验

研究生：徐宇鸿

提交日期：2026年05月06日

研究生签名：

学号	202267330269	学院	计算机科学与工程学院
课程编号		课程名称	软件测试与质量控制
学位类别	☐ 博士 ☐ 硕士	任课教师	李方
学习形式	☐ 全日制 ☐ 非全日制	学期	2025-2026 学年第二学期

教师评语：

成绩评定：

分

任课教师签名：

年

月

日

移动软件测试工具体验

课程报告

Yuhong Xu

1. 作业概述

本作业是软件测试与质量控制课程的移动软件测试工具体验环节，要求体验一款移动端崩溃与异常监控工具，在平台上注册应用并集成 SDK，打包后在移动终端运行，再由平台给出程序异常、崩溃和用户行为的分析报告。课程报告需要附上主要操作过程、主要源代码，以及网站测试中发现的异常或不好用的功能点。

本次体验选用腾讯 Bugly 作为移动测试工具。它是国内主流的移动应用质量监控平台，能够采集和分析 Java 崩溃、Native 崩溃、ANR 与各类运行时异常。被测对象是本课程中自行开发的旅行攻略前后台综合应用，由一个 Android 客户端和一个 Java Spring Boot 后端组成，满足作业对源代码、后台 Java 实现和前台 Android 的全部要求，也覆盖了加分项的前后台综合应用。

2. 被测的前后台综合应用

2.1. 系统架构

被测系统采用移动客户端加 RESTful 后端加嵌入式数据库的三层结构，整体如图 1 所示。Android 客户端通过 Retrofit 发起 HTTP 请求与后端交互，后端基于 Spring Boot 实现，使用 H2 嵌入式数据库持久化数据，崩溃与异常数据则由集成在客户端中的 Bugly SDK 直接上报到腾讯云端。

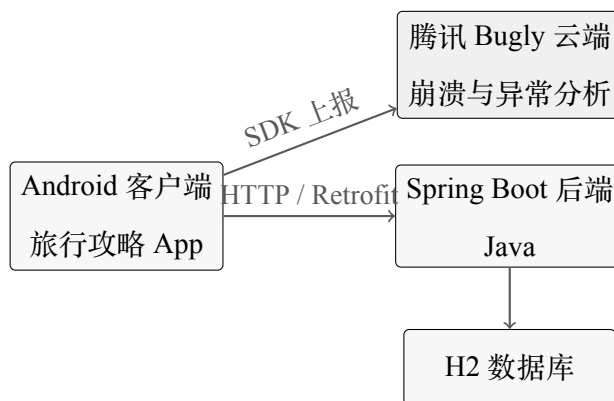


图 1 被测前后台综合应用系统架构

该系统同时承担两类被测目标。一类是后端 Java 进程在处理请求时抛出的运行时异常，由后端的 `TestExceptionHandler` 主动触发，再由 `GlobalExceptionHandler` 统一捕获。另一类是 Android 客户端运行时发生的崩溃，包括 Java 崩溃、Native 崩溃和 ANR，由客户端的 `CrashTester` 主动触发，再由 Bugly SDK 自动采集上报。

2.2. 后台 Java 实现

后端基于 Spring Boot 3 构建。业务功能由目的地、攻略、用户等一组控制器提供，异常测试集中在 `TestExceptionHandler`，统一异常处理在 `GlobalExceptionHandler`，用户行为的记录与查询在 `UserBehaviorController`。数据通过 Spring Data JPA 的仓库接口持久化到 H2 数据库。

后台启动后默认监听 8080 端口，并暴露 H2 数据库的 Web 控制台，方便直接查看数据。访问 `http://localhost:8080/h2-console` 即可登录数据库查看行为记录和业务数据，如图 2 所示。



图 2 后端 H2 数据库的 Web 控制台登录页

2.3. 前台 Android 实现

Android 客户端包含登录注册、目的地列表、攻略详情、搜索、点赞等功能页面，使用 Retrofit 加 Gson 与后端通信。为支持移动端测试工具体验，客户端额外实现了两个工具类。`CrashTester` 用于主动触发各类崩溃，用来验证 SDK 的采集能力。`BehaviorTracker` 用于记录用户在 App 内的页面访问、点击、搜索等行为并异步上报后端。应用入口 `TravelGuideApplication` 中完成 Bugly SDK 的初始化。

3. 移动测试工具的集成

3.1. 工作模式与选型

移动端崩溃监控类工具的工作模式大致相同。先在平台创建应用拿到 `AppId`，再把 SDK 集成进客户端工程，打包出带 SDK 的安装包并在设备上运行，SDK 会自动把崩溃、异常、性能和行为数据上报回平台，最后登录网站查看分析报告。腾讯 Bugly 完整

覆盖了这条链路，提供崩溃、异常、ANR 的采集与分析，以及趋势统计和告警能力，因此选作本次体验的工具。

3.2. SDK 集成

客户端通过 Gradle 引入腾讯 Bugly 的崩溃采集 SDK，依赖配置见代码 1。随后在应用入口 `TravelGuideApplication.onCreate()` 中调用 `CrashReport.initCrashReport()` 完成初始化，见代码 2。初始化的第三个参数是调试模式开关，调试模式下 SDK 会输出详细日志并加快上报，便于实验阶段快速观察采集结果。

列表 1 Bugly SDK 的 Gradle 依赖

```
dependencies {
    // 腾讯 Bugly 崩溃采集 SDK
    implementation 'com.tencent.bugly:crashreport:4.1.9.3'
    // Retrofit 用于与后端通信
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
    // ...其余依赖省略
}
```

列表 2 Bugly SDK 的初始化代码

```
public class TravelGuideApplication extends Application {
    private static final String BUGLY_APP_ID = "3f957d9605";

    @Override
    public void onCreate() {
        super.onCreate();
        try {
            // 第三个参数为调试模式开关，true 时输出详细日志并加快上报
            CrashReport.initCrashReport(this, BUGLY_APP_ID, true);
            Log.i("TravelGuideApp", "Bugly SDK initialized, appId=" + BUGLY_APP_ID);
        } catch (Throwable t) {
            Log.e("TravelGuideApp", "Bugly SDK initialization failed", t);
        }
    }
}
```

4. 测试体验与实验过程

本节按后台 Java 异常捕获、移动端崩溃采集、用户行为分析三条线给出主要操作过程和主要源代码，对应作业要求的三类分析报告。

4.1. 后台 Java 异常捕获

为验证工具对后台 Java 异常的捕获能力，后端实现了 `TestExceptionHandler`，提供一组专门用于触发各类运行时异常的 HTTP 接口。访问 `/api/test` 可以获取全部异常测试接口列表，如图 3 所示，共 7 个接口，分别对应空指针、数组越界、除零、数字

格式、资源未找到、业务异常和非法参数异常。

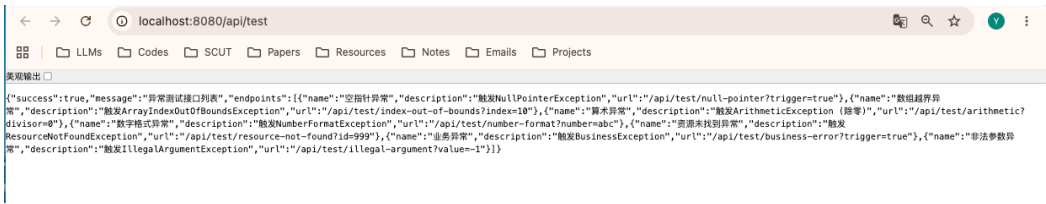


图 3 后台 /api/test 返回的异常测试接口列表

各接口的触发参数和预期异常见表 1。以空指针和除零两个接口为例，其实现见代码 3。当请求携带特定参数时，方法体内会故意执行会抛出异常的操作。

表 1 后台异常测试接口一览

接口路径	触发参数	预期异常
/api/test/null-pointer	trigger=true	NullPointerException
/api/test/index-out-of-bounds	index=10	ArrayIndexOutOfBoundsException
/api/test/arithmetic	divisor=0	ArithmeticException
/api/test/number-format	number=abc	NumberFormatException
/api/test/resource-not-found	id=999	ResourceNotFoundException
/api/test/business-error	trigger=true	BusinessException
/api/test/illegal-argument	value=-1	IllegalArgumentException

列表 3 主动触发异常的接口节选

```
@GetMapping("/null-pointer")
public ResponseEntity<Map<String, Object>> testNullPointerException(
    @RequestParam(required = false, defaultValue = "false") boolean trigger) {
    if (trigger) {
        String nullString = null;
        int length = nullString.length(); // 抛出 NullPointerException
    }
    Map<String, Object> response = new HashMap<>();
    response.put("success", true);
    response.put("message", "没有触发异常");
    return ResponseEntity.ok(response);
}

@GetMapping("/arithmetic")
public ResponseEntity<Map<String, Object>> testArithmeticException(
    @RequestParam(defaultValue = "1") int divisor) {
    int result = 100 / divisor; // divisor 为 0 时抛出 ArithmeticException
    // ...
}
```

这些异常由 `GlobalExceptionHandler` 统一捕获。它用 `@RestControllerAdvice` 注册一组 `@ExceptionHandler`，把异常转换成统一格式的错误响应返回给调用方，同时调用 `ex.printStackTrace()` 把完整堆栈打印到后台控制台，供测试工具或运维捕获，见代码 4。

列表 4 全局异常处理器节选

```
@ExceptionHandler(NullPointerException.class)
public ResponseEntity<ErrorResponse> handleNullPointerException(
    NullPointerException ex, WebRequest request) {
    ErrorResponse errorResponse = new ErrorResponse(
        HttpStatus.INTERNAL_SERVER_ERROR.value(), "空指针异常",
        "系统内部错误: " + ex.getMessage(),
        request.getDescription(false), LocalDateTime.now());
    ex.printStackTrace(); // 打印堆栈，供测试工具捕获
    return new ResponseEntity<>(errorResponse, HttpStatus.INTERNAL_SERVER_ERROR);
}
```

4.2. 移动端崩溃采集

移动端通过 `CrashTester` 在主界面菜单中弹出选择框，可以主动触发不同类型的崩溃和异常，用来验证 Bugly 的采集能力。可选类型包括自定义的空指针、数组越界、除零三类 Java 崩溃，Bugly 官方内置的 Java、Native、ANR 崩溃测试，以及主动上报已捕获异常且不导致崩溃的一种方式，见代码 5。前三类自定义崩溃没有做 try-catch，会直接导致 App 崩溃并被 Bugly 自动采集。最后一种用 `CrashReport.postCaughtException()` 上报已经被 catch 的异常，App 不会崩溃，但同样能在平台看到记录。

列表 5 移动端崩溃的触发代码节选

```
private static void triggerNullPointer() {
    String nullStr = null;
    int length = nullStr.length(); // 未捕获，导致崩溃，由 Bugly 采集
}

private static void triggerArithmetic() {
    int a = 10, b = 0;
    int c = a / b; // 除零，触发 ArithmeticException
}

private static void reportCaughtException() {
    try {
        Integer.parseInt("not_a_number");
    } catch (Exception e) {
        CrashReport.postCaughtException(e); // 捕获后上报，App 不崩溃
    }
}
```

实验的操作过程是这样的。先启动后端 `./gradlew bootRun` 并保持运行，再用 Android Studio 把客户端安装到模拟器，依次进入主界面菜单的崩溃测试，选择不同条

目触发崩溃。App 崩溃重启后，Bugly SDK 会把崩溃堆栈异步上报到云端，之后登录 Bugly 控制台即可查看采集结果。

4.3. 用户行为分析

客户端的 BehaviorTracker 以单例形式记录用户行为，定义了页面进入与退出、点击、搜索、分享、点赞、登录登出与注册等行为类型。每当用户执行相应操作，就调用 trackBehavior() 把行为类型、行为详情、页面名、会话 ID 和设备信息组装起来，通过 Retrofit 异步上报到后端的 /api/behavior/record 接口，见代码 6。

列表 6 用户行为的记录与上报节选

```
public void trackBehavior(String actionType, String actionDetail, String pageName) {
    Map<String, Object> behaviorData = new HashMap<>();
    behaviorData.put("userId", userId);
    behaviorData.put("actionType", actionType);
    behaviorData.put("actionDetail", actionDetail);
    behaviorData.put("pageName", pageName);
    behaviorData.put("sessionId", sessionId);
    behaviorData.put("deviceInfo", getDeviceInfo());
    // 异步上报到后端
    apiService.recordBehavior(behaviorData).enqueue(new Callback<>() { /* ... */ });
}
```

后端 UserBehaviorController 接收并持久化这些行为记录，并提供按用户、按会话、按行为类型、按时间段的查询接口。其中 GET /api/behavior/session/{sessionId} 会返回某会话内的全部行为及其页面访问顺序，对应作业要求的记录访问发掘页面顺序。

5. 实验结果与分析

5.1. 移动端崩溃采集结果

登录腾讯 Bugly 控制台后，在异常概览页面可以看到本次实验触发的崩溃记录，如图 4 所示。列表中每条记录包含异常编号、异常类型、发生次数、影响用户数、发生时间和异常摘要。本次实验共采集到 5 条记录，覆盖 Java 层和 Native 层两类崩溃，汇总见表 2。

点击列表中的任一条目可以进入异常详情页，查看完整的堆栈和线程信息。以 Native 崩溃 SIGABRT 为例，详情页展示了带内存偏移地址的 Native 调用栈，能定位到 libart.so、libgui.so 等系统库中的崩溃位置，如图 5 所示。这说明 Bugly 不仅采集 Java 层异常，还能还原 Native 层崩溃，后续上传符号表后就可以把这些内存地址解析成可读的源码行号。

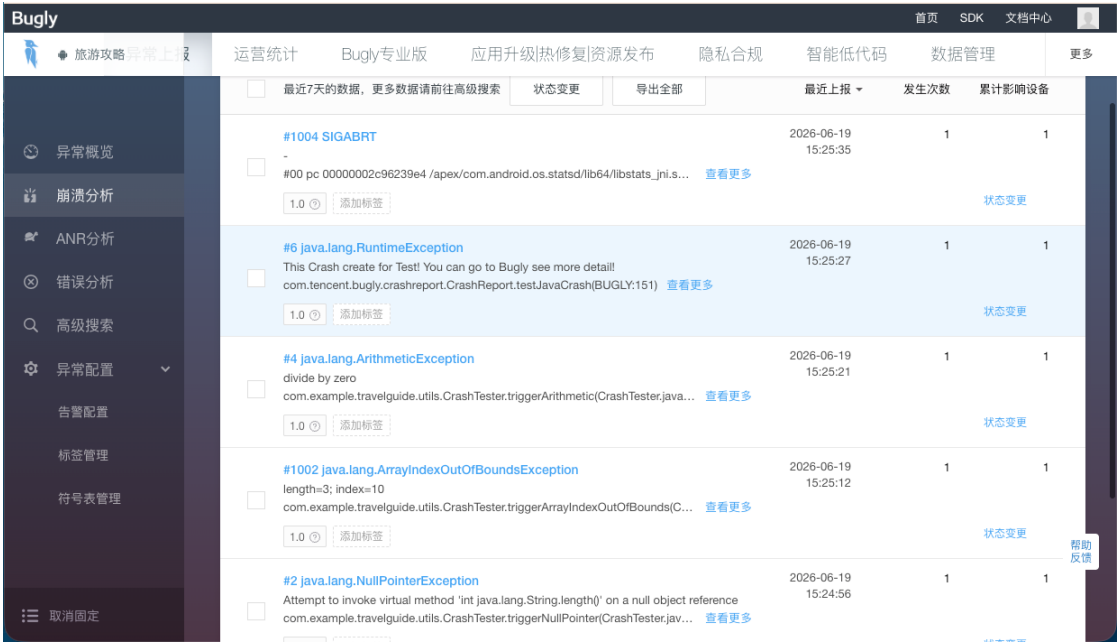


图 4 腾讯 Bugly 的异常概览列表

表 2 Bugly 采集到的崩溃与异常记录

异常类型	触发方式与异常摘要
SIGABRT	由 testNativeCrash() 触发的 Native 层崩溃
RuntimeException	由 testJavaCrash() 触发的官方测试崩溃
ArithmeticException	由 triggerArithmetic() 触发的除零异常
ArrayIndexOutOfBoundsException	由 triggerArrayIndexOutOfBounds() 触发的数组越界
NullPointerException	由 triggerNullPointer() 触发，对 null 调用 length()

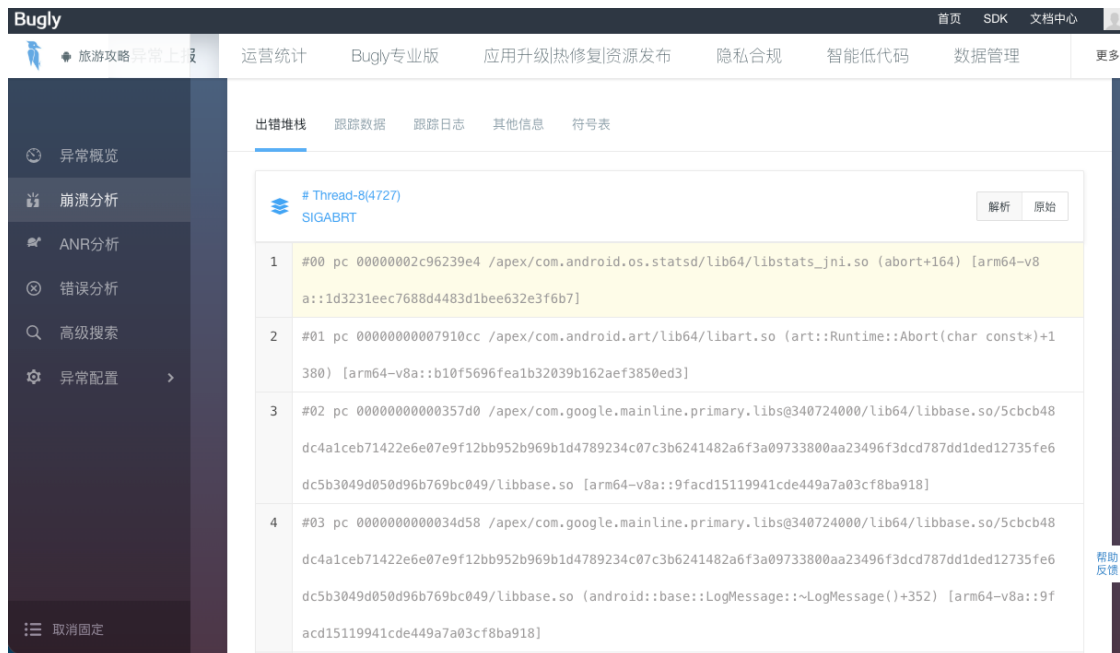


图 5 腾讯 Bugly 的异常堆栈详情

5.2. 结果分析

从采集结果可以得出几点结论。第一，自定义触发的三类 Java 崩溃，也就是空指针、数组越界和除零，都被正确采集，异常类型和堆栈定位到的 `CrashTester` 方法一一对应，验证了 SDK 的崩溃捕获与堆栈还原能力。第二，Bugly 官方提供的 Java、Native、ANR 测试接口同样被记录，其中 Native 崩溃能还原出完整的系统库调用栈，说明工具具备跨 Java 层和 Native 层的采集能力。第三，`postCaughtException()` 上报的已捕获异常也出现在记录中，并且不会导致 App 崩溃，满足记录非致命异常的需求。

6. 网站测试异常与不好用的功能点

本节汇总使用腾讯 Bugly 过程中发现的不好用的功能点和需要注意的问题，对应作业要求中的这一项。

6.1. Native 崩溃堆栈需要符号表才能定位

本次实验采集到的 SIGABRT 属于 Native 层崩溃。在异常详情页可以看到，这类崩溃上报的是带内存偏移地址的调用栈，比如 `libart.so` 和 `libgui.so` 中的 pc 地址，无法直接读出函数名和源码行号。要把它解析成可读信息，需要先用 NDK 编译出带调试符号的 so 文件，再把符号表上传到 Bugly 的符号表管理页面。这意味着 Native 崩溃的定位成本明显高于 Java 崩溃，是这类工具普遍存在的使用门槛。本次实验以 Java 崩溃为主，Native 崩溃由官方测试接口触发，影响不大，但在真实项目里处理 so 崩溃时，符号表的准备和上传是不可省略的一步。

6.2. 数据上报存在延迟

Bugly SDK 采用缓存加批量上报的方式，不会在崩溃发生的瞬间就把数据传到云端。即使初始化时开启了调试模式来加快上报，操作完成后通常也要等几分钟，刷新控制台才能看到新的崩溃记录。实验过程中第一次触发崩溃后立即去网站查看，列表是空的，一度让人怀疑采集没有成功，实际上只是上报还没有完成。这一点在使用时需要心中有数，排查问题的时候不能因为网站暂时没有数据就否定 SDK 的工作。

6.3. 调试模式与发布模式需要区分

初始化代码里第三个参数设为 true，开启的是调试模式，会输出详细日志并加快上报，方便实验阶段观察。这个模式不适合直接带到正式发布版本，详细日志会带来额外的性能开销，也可能在日志里暴露敏感信息。正式打包前需要把这个参数改成 false，或者按 Build 类型区分。这类配置切换容易在赶进度时被忽略，属于集成时需要留意的一个功能点。

7. 总结

本次作业围绕移动软件测试工具体验展开，以自研的旅行攻略前后台综合应用为被测对象，完整走通了集成 SDK、触发异常与崩溃、上报数据、登录平台查看分析报告的全流程。后台 Java 异常通过 `TestExceptionHandler` 和 `GlobalExceptionHandler` 触发并捕获，移动端崩溃通过 `CrashTester` 触发并由 Bugly SDK 采集，用户行为通过 `BehaviorTracker` 上报后端并支持会话级的页面访问顺序查询，三类分析报告都得到了实际数据的支撑。

在腾讯 Bugly 上，作业的核心目标，也就是移动端崩溃、异常和 ANR 的采集与分析顺利达成，采集到的 Java 崩溃、Native 崩溃和已捕获异常记录都能在控制台正确还原堆栈。使用中也发现，Native 崩溃需要符号表才能定位、数据上报存在延迟、调试与发布模式需要区分，这几点是实际落地时要关注的。

通过这次作业，我对移动端崩溃监控工具的工作模式有了直观的认识。这类工具的价值在于用嵌入 SDK 加云端聚合的方式，把分散在大量设备上的崩溃和行为数据集中成可分析、可追溯的报告，帮助开发者快速定位稳定性问题。同时也体会到，工具选型不能只看功能列表，平台的可用性比如证书维护和接口稳定性，同样是能否实际落地的关键前提。

A. 后台核心代码

A.1. 用户行为服务层

用户行为服务层是后端处理用户行为的核心，负责把客户端上报的行为落库，并提供按用户、会话、类型和时间段的查询。其中 `getUserPageSequence` 方法从某次会话的全部行为里筛出页面进入记录，按时间顺序拼成访问路径，对应作业要求的记录访问发掘页面顺序。

列表 7 用户行为的记录与页面顺序拼接

```
@Transactional
public UserBehavior recordBehavior(Long userId, String actionType, String actionDetail
    ,
                                String pageName, String sessionId, String
deviceInfo) {
    UserBehavior behavior = new UserBehavior();
    if (userId != null) {
        behavior.setUser(userService.getUserById(userId)); // 关联用户
    }
    behavior.setActionType(actionType); // 行为类型, 如 PAGE_ENTER、CLICK
    behavior.setPageName(pageName);
    behavior.setSessionId(sessionId); // 会话 id, 串起一次访问的全部行为
    return behaviorRepository.save(behavior); // 写入 H2 数据库
}

// 拼接某次会话的页面访问顺序
public String getUserPageSequence(String sessionId) {
    List<UserBehavior> behaviors = getSessionBehaviors(sessionId);
    StringBuilder sequence = new StringBuilder();
    for (UserBehavior behavior : behaviors) {
        if (ACTION_PAGE_ENTER.equals(behavior.getActionType())) { // 只取页面进入
            if (sequence.length() > 0) sequence.append(" -> ");
            sequence.append(behavior.getPageName());
        }
    }
    return sequence.toString();
}
```

A.2. 数据初始化

数据初始化在应用启动时通过 `CommandLineRunner` 往数据库灌入测试数据，包括两个用户、五个目的地和两条攻略。这样 App 一启动就有内容可以浏览、搜索和点赞，用户行为数据才有来源。每张表都用 `count() == 0` 判断，只在表为空时灌入，避免重复启动时重复写入。

列表 8 启动时灌入测试数据

```
@Bean
CommandLineRunner initData(UserRepository userRepository,
                           DestinationRepository destinationRepository,
                           TravelGuideRepository guideRepository) {
```

```

return args -> {
    if (userRepository.count() == 0) {           // 表为空才初始化
        userRepository.save(new User("testuser", "123456", "test@example.com"));
    }
    if (destinationRepository.count() == 0) {
        Destination d1 = new Destination("北京", "中国首都, 历史悠久", "中国");
        d1.setRating(4.8);
        destinationRepository.save(d1);          // 北京、上海、广州、成都、杭州
    }
    // 攻略同理, 并通过 setDestination 关联到对应目的地
};
}

```

B. 前台核心代码

B.1. 网络通信封装

网络通信用一个单例类封装 Retrofit 实例。其中 BASE_URL 设为 `http://10.0.2.2:8080/`, 这是 Android 模拟器访问宿主机 localhost 的特殊地址。OkHttpClient 配了 30 秒超时和 BODY 级别的日志拦截器, 调试时能看到完整的请求和响应内容, 对排查行为上报是否成功很有帮助。

列表 9 Retrofit 的单例封装

```

public class ApiClient {
    // 模拟器访问宿主机 localhost 的特殊地址
    private static final String BASE_URL = "http://10.0.2.2:8080/";
    private static ApiClient instance;
    private final ApiService apiService;

    private ApiClient() {
        HttpLoggingInterceptor logging = new HttpLoggingInterceptor();
        logging.setLevel(HttpLoggingInterceptor.Level.BODY); // 打印完整请求体
        OkHttpClient client = new OkHttpClient.Builder()
            .connectTimeout(30, TimeUnit.SECONDS)
            .addInterceptor(logging).build();
        Retrofit retrofit = new Retrofit.Builder()
            .baseUrl(BASE_URL).client(client)
            .addConverterFactory(GsonConverterFactory.create()).build();
        apiService = retrofit.create(ApiService.class); // 动态生成接口实现
    }
    public static synchronized ApiClient getInstance() { /* 单例 */ }
}

```

B.2. API 接口定义

所有后端接口用一个 Java 接口声明, 借助 Retrofit 的注解把 HTTP 方法、路径和参数描述清楚, 分成用户、目的地、攻略、行为和测试异常五组。其中行为和测试异常两组直接服务于本作业。ApiResponse 是统一的响应包装, 包含 success、message 和 data 三个字段, 客户端据此判断调用是否成功。

列表 10 后端接口的声明节选

```

public interface ApiService {
    // 上报用户行为，供 BehaviorTracker 调用
    @POST("api/behavior/record")
    Call<ApiResponse<UserBehavior>> recordBehavior(@Body Map<String, Object>
        behaviorData);

    // 查询某会话的全部行为，用于还原页面访问顺序
    @GET("api/behavior/session/{sessionId}")
    Call<ApiResponse<List<UserBehavior>>> getSessionBehaviors(@Path("sessionId")
        String sessionId);

    // 触发后台除零异常，供测试调用
    @GET("api/test/arithmetic")
    Call<ApiResponse<Object>> testArithmeticException(@Query("divisor") int divisor);

    // 统一响应包装
    class ApiResponse<T> {
        private boolean success;
        private String message;
        private T data;
    }
}

```

B.3. 会话管理

会话管理用 SharedPreferences 保存登录态和用户信息，App 关掉重开仍然保持登录。createLoginSession 在登录成功后写入，logout 清空全部信息。行为追踪里的 userId 也从这里取，让上报的行为能关联到具体用户。

列表 11 登录态的本地保存

```

public class SessionManager {
    // 登录成功后写入登录态和用户信息
    public void createLoginSession(Long userId, String username, String nickname,
        String email) {
        editor.putBoolean(KEY_IS_LOGGED_IN, true);
        editor.putLong(KEY_USER_ID, userId);
        editor.putString(KEY_USERNAME, username);
        editor.apply(); // 异步落盘
    }

    public boolean isLoggedIn() {
        return preferences.getBoolean(KEY_IS_LOGGED_IN, false);
    }

    public void logout() { editor.clear(); editor.apply(); } // 登出清空
}

```